

**DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING**

Course Code: CS-116

Course Title: Object Oriented Programming

Complex Engineering Problem

FE Batch 2025, Spring Semester 2026

Grading Rubric

TERM PROJECT

Student No.	Name	Roll No.
S1	Daniyal Ahmed	CS-25134
S2	Muhammad Shariq	CS-25140
S3	Hayyan Khan	CS-25147
S4	Sufyan Khan	CS-25135

CRITERIA AND SCALES					Marks Obtained			
					S1	S2	S3	S4
Criterion 1: Does the application meet the desired specifications and produce the desired outputs? (CPA-1, CPA-3)								
1	2	3	4					
The application does not meet the desired specifications and is producing incorrect outputs.	The application partially meets the desired specifications and is producing incorrect or partially correct outputs.	The application meets the desired specifications but is producing incorrect or partially correct outputs.	The application meets all the desired specifications and is producing correct outputs.					
Criterion 2: How well is the code organization?								
1	2	3	4					
The code is poorly organized and very difficult to read.	The code is readable only to someone who knows what it is supposed to be doing.	Some part of the code is well organized, while some part is difficult to follow.	The code is well organized and very easy to follow.					
Criterion 3: How friendly is the application interface? (CPA-1, CPA-3)								
1	2	3	4					
The application interface is difficult to understand and use.	The application interface is easy to understand and but not that comfortable to use.	The application interface is very easy to understand and use.	The application interface is very interesting/ innovative and easy to understand and use.					
Criterion 4: How does the student performed individually and as a team member? (CPA-2, CPA-3)								
1	2	3	4					
The student did not work on the assigned task.	The student worked on the assigned task, and accomplished goals partially.	The student worked on the assigned task, and accomplished goals satisfactorily.	The student worked on the assigned task, and accomplished goals beyond expectations.					
Criterion 5: Does the report adhere to the given format and requirements?								
1	2	3	4					
The report does not contain the required information and is formatted poorly.	The report contains the required information only partially but is formatted well.	The report contains all the required information but is formatted poorly.	The report contains all the required information and completely adheres to the given format.					
Total Marks:								

1. Problem Description

The CIS Shelf E-Library Management System is a console-based application developed in C++ to address the operational challenges faced by academic and institutional libraries. Traditional library management relies heavily on manual record-keeping, which is prone to errors, time-consuming, and inefficient in tracking borrowing activities, managing resource availability, and enforcing return deadlines. This project provides a fully automated, software-driven solution to overcome these problems.

The primary objective of the system is to automate key library operations, including user registration and authentication, resource cataloguing, borrowing and return processing, overdue fine calculation, and administrative reporting. The system maintains permanent data persistence through file-based storage, ensuring that all records — user accounts, library resources, and borrowing transactions — are preserved across sessions without the need for an external database management system.

The intended users of this system are twofold: library members (students or general users) who interact with the system to browse available resources, borrow and return books, and monitor their fine status; and library administrators (librarians) who manage the resource inventory and review user activity reports. The system is designed to serve an academic institution's Computer and Information Systems department, offering a scalable and maintainable foundation that can be extended into a fully networked or graphically enhanced application in the future. By integrating a Python-based graphical user interface alongside the C++ core backend, the project also demonstrates a practical cross-language software architecture suitable for real-world deployment.

2. Distinguishing Features of the Project / Object-Oriented Features Implemented

CIS Shelf distinguishes itself from a basic file-management program by implementing a rich set of Object-Oriented Programming (OOP) principles within a meaningful, real-world context. The following subsections describe each concept as applied in this project.

2.1 Abstraction and Abstract Classes

An abstract base class, `LibraryEntity`, serves as the root of the class hierarchy. It declares a pure virtual function `Display() const = 0`, making it impossible to instantiate directly. All derived classes — `Resource` and `User` — must provide their own concrete implementation of `Display()`, enforcing a uniform interface across all library entities. A virtual destructor is also declared in the base class, following best practice for polymorphic class hierarchies.

2.2 Inheritance

Both the Resource and User classes publicly inherit from LibraryEntity. This inheritance relationship models the is-a semantic correctly: a Resource is a LibraryEntity, and a User is a LibraryEntity. Inheritance allows the manager classes to interact with these entities through a common interface, and it avoids code duplication by centralising shared structural requirements in the base class.

2.3 Polymorphism and Method Overriding

Polymorphism is realised through method overriding. Both Resource and User override the pure virtual Display() method from LibraryEntity, each printing information relevant to their own domain. The override keyword is used explicitly on both overriding methods to prevent accidental signature mismatches. When UserManager::printAllUsers() iterates the user collection and calls Display() on each object, it exercises runtime polymorphism through the overridden method.

2.4 Encapsulation and Data Hiding

All data members across every class — IDs, titles, passwords, balances, borrow counts, issue and due dates — are declared private. Public getter and setter methods provide controlled, validated access. For example, User::deduct_fine() enforces a balance check before modifying the balance field, and Resource::setAvailability() is the only external means of changing a resource's borrow status. This strict encapsulation prevents data corruption from arbitrary external modification.

2.5 Operator Overloading

The Resource class overloads the equality operator (==) in two forms: one that compares two Resource objects by their internal ID, and one that compares a Resource object directly against an integer. This allows BorrowManager and ResourceManager to write natural, readable lookup code such as *if (resource == id)* instead of the more verbose *if (resource.getId() == id)*, improving both readability and expressiveness.

2.6 Constructor Overloading

Both Borrow_Record and User define multiple constructors to support different creation contexts. Borrow_Record has a constructor for live borrow operations (which automatically records the current time and calculates the due date) and a second constructor for reconstructing records from saved files (which accepts explicit timestamps). User similarly has one constructor for new registrations (applying default balance) and another for file-loaded records (restoring exact saved values).

2.7 Exception Handling

Structured exception handling using try-catch blocks is applied at every point where user input conversion or system resource access could fail. stoi() calls for resource and menu IDs are wrapped to catch invalid_argument exceptions, preventing crashes from non-numeric input. Fine_calculation() throws a runtime_error if the system clock returns -1. deduct_fine() throws a runtime_error if the outstanding fine exceeds the user's balance, triggering the defaulter warning flow without corrupting the balance.

2.8 Composition (Has-A Relationship)

The `Library_Management` class owns concrete instances of `ResourceManager`, `UserManager`, and `BorrowManager` as private member objects. This composition (has-a) relationship means the toplevel controller is assembled from three fully functional subsystems, each responsible for its own domain. The UML class diagram confirms this with 1-to-1 composition arrows from `Library_Management` to each manager class.

2.9 File Handling and Data Persistence

All three manager classes use C++ standard file streams (`ifstream` and `ofstream`) to load and save data in pipe-delimited plain-text format across three files: `resources.txt`, `users.txt`, and `records.txt`. On startup, `loadAllData()` restores the complete in-memory state from disk. After every write operation — registration, borrow, return, add, or delete — the relevant manager immediately flushes its vector to disk via `saveToFile()` (exposed as `syncFile()` for cross-class calls), ensuring zero data loss even if the program is closed between transactions.

2.10 Dynamic Arrays with STL Vectors

Each manager class maintains its collection of entities as a `std::vector`, providing automatic dynamic resizing, $O(1)$ amortised appending via `emplace_back()`, and range-based iteration. This eliminates manual memory management and makes the codebase substantially safer and more maintainable than raw pointer arrays would be.

2.11 Time-Based Fine Calculation

The `ctime` library is used to capture issue timestamps (`time(0)`), compute due dates (`issueDate + 604800` seconds = 7 days), and calculate overdue fines. The fine formula applies a rate of Rs. 10 per fractional day of delay, meaning even a partial day's overdue incurs a proportional charge. The system also formats timestamps for human-readable display using `strftime`.

2.12 Menu-Driven Architecture

The system provides two independent menu-driven interfaces: a User Menu offering six operations (view resources, search by title, borrow, return, check fine, logout) and an Admin Panel with five operations (add resource, delete resource, view all users, view all records, exit). Each menu loop validates input using `cin.fail()` checks and re-prompts on invalid entries.

3. Flow of the Project Including Class Diagram

The CIS Shelf system follows a layered, top-down architectural flow. On startup, the `main()` function instantiates a single `Library_Management` object and delegates all control to its `mainMenu()` method. The very first action of `mainMenu()` is `loadAllData()`, which calls `loadFromFile()` on all three managers in sequence — `ResourceManager`, `UserManager`, and `BorrowManager` — restoring the complete application state from disk into their respective vectors. If no resources exist (first run), `addDefaultResources()` seeds the system with five default books.

The main menu loop presents four options. Selecting Register invokes

`UserManager::register_user()`, which collects six fields, appends a new `User` to the vector, saves to disk, and dispatches a Python-driven confirmation email. Selecting Login invokes `UserManager::login_user()`, which searches the user vector for a matching username and password; on success, it returns the username string to `mainMenu()`, which then calls `findUser()` to obtain a live pointer to the `User` object and passes it to `userMenu()`. The user menu pointer is re-fetched at the top of each iteration to reflect any in-session data changes. Selecting Librarian Panel opens `adminMenu()` without any authentication gate.

During a borrow operation, `BorrowManager::borrow_resource()` receives the `User` pointer, the shared resource vector, and references to both `UserManager` and `ResourceManager`. It checks the user's borrow count, locates the resource by ID using the overloaded operator `==`, marks it unavailable, creates a `Borrow_Record` with the current timestamp and a 7-day due date, and calls `syncFile()` on all three managers. During return, the reverse process unfolds: the matching active record is located, the fine is calculated and deducted (or a defaulter alert is raised), the record is marked returned, the resource is marked available, and all files are synced.

The UML class diagram below illustrates the complete class structure, including inheritance arrows from `Resource` and `User` to the abstract `LibraryEntity` base class, composition arrows from `Library_Management` to the three manager classes, and one-to-many association arrows from each manager to its corresponding entity class.

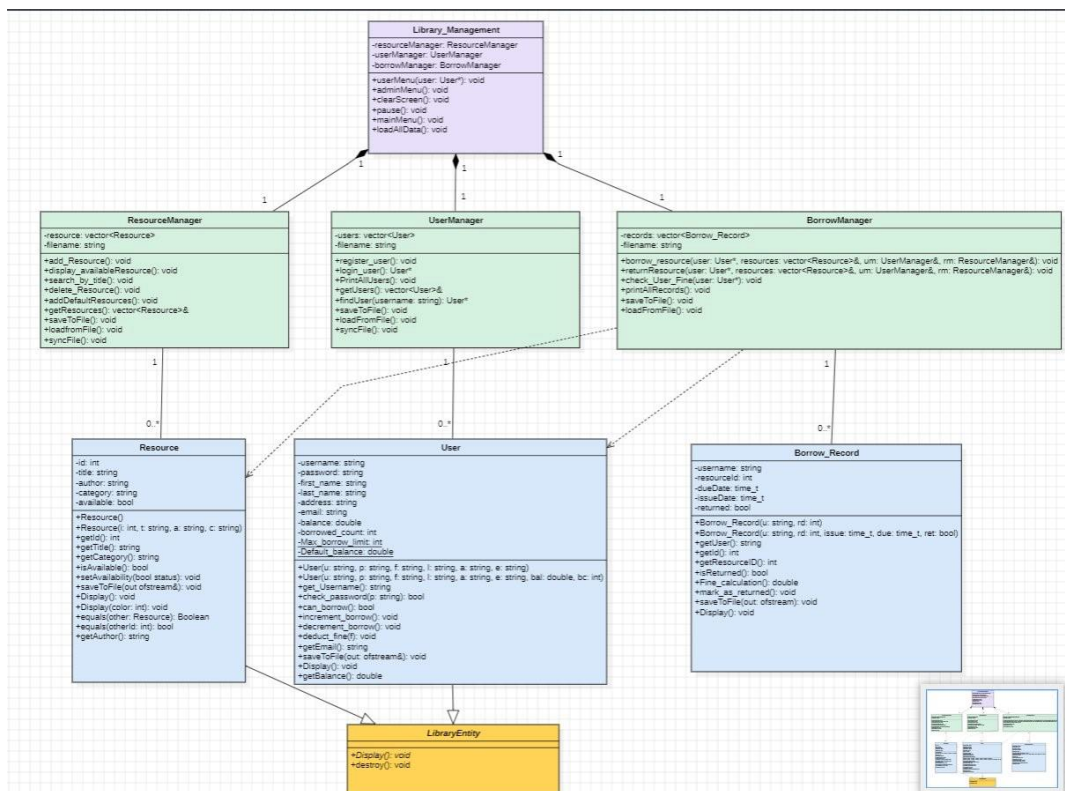


Figure 1: UML Class Diagram — CIS Shelf E-Library System

4. Most Challenging Part While Working on the Project

Each team member encountered distinct technical and logical challenges.

Daniyal Ahmed (CS-25134) — Graphical User Interface Development

The most challenging aspect for Daniyal was designing and implementing the Graphical User Interface (GUI) using Python and seamlessly connecting it to the C++ backend. First I tried the Qt GUI but Qt takes a lot of time to develop and it need a proper development. Yes, I reviewed Qt as well but I was lil frustrated to see that. Then I decided to develop a proper GUI using Python cause I used Python before for some Ai automation projects.

Muhammad Shariq (CS-25140) · User and UserManager Modules

Shariq found the most challenging part to be designing the authentication logic and ensuring user data was safely encapsulated and correctly persisted across sessions without duplication. He also learned to structure classes so that sensitive data is never exposed directly, reinforcing the principle of least privilege across the entire user module.

Hayyan Khan (CS-25147) · ResourceManager and BorrowRecord Module

Hayan's greatest challenge was implementing the case-insensitive substring search and ensuring that resource deletion was safely blocked whenever a book was currently borrowed by any user. He further discovered how to write defensive deletion logic that queries cross-module state before committing irreversible operations, making the system robust under concurrent usage.

Sufyan Ramzan (CS-25135) · BorrowManager Module

Sufyan found that the aspect to be correctly implementing the cross-class borrow and return logic, which required careful management of shared state across the User, Resource, and Borrow_Record objects while keeping all three files in sync atomically.

5. New Things Learned While Working on the Project

Daniyal Ahmed (CS-25134)

I developed a proper and a unique mindset that how can I think different and make our project unique. I first thought that we will definitely fulfill the requirements that we have setup but the main thing is that was our project must stand out to be unique. I practiced that how we can connect our system with a proper pipeline of registering to email and then pdf.

Muhammad Shariq (CS-25140)

Shariq deepened his understanding of how authentication systems work internally and how file persistence enables data continuity across program sessions without a database. He also gained confidence in implementing serialisation and deserialisation routines from scratch, understanding exactly how structured text files map to in-memory object states.

Hayyan Khan (CS-25147)

Hayan learned how STL vectors simplify dynamic data management and how modular class design makes a codebase significantly more scalable and maintainable. He additionally practised breaking down a large problem into independently testable units, which significantly reduced integration bugs during the final assembly phase.

Sufyan Ramzan (CS-25135)

Sufyan gained practical experience in time-based processing using the ctime library and understood how complex business rules · such as fine calculation and borrow limits · are

translated into reliable code logic. He also strengthened his ability to reason about concurrency risks in single-threaded applications, developing defensive coding patterns that prevent silent data corruption across dependent modules.

6. Individual Contributions of Each Member

The project was developed collaboratively, with each member assigned a primary area of responsibility. The CIS Shelf AI Assistant chatbot feature was developed through joint cooperation by all four members. The table below summarises each member's specific contributions.

Name	Roll No.	Contributions & Responsibilities
Daniyal Ahmed	CS-25134	Python GUI (graphical front-end), email notification system, LibraryEntity abstract base class, Resource class, Library_Management class.
Muhammad Shariq	CS-25140	User class (encapsulation, constructors, balance management, borrow limit enforcement), UserManager class (registration, login, file persistence, findUser lookup).
Hayyan Khan	CS-25147	ResourceManager class (add, delete, search by title, display available resources, default resource seeding, file I/O synchronisation). Borrow Record class(Fine calculation, mark as returned, display)
Sufyan Ramzan	CS-25135	BorrowManager class (borrow and return logic, defaulter detection, fine checking, borrow record persistence, cross-class state management).
All Members	All	CIS Shelf AI Assistant chatbot (integrated into the GUI, jointly designed and implemented through group cooperation).

While CIS Shelf successfully delivers all core library management functionalities, several significant enhancements are planned for future development iterations.

FUTURE EXPANSIONS :

7.1 Full-Featured Connected GUI

The current Python GUI handles registration and launches the C++ console application as a subprocess. A future version will replace the console entirely with a fully integrated graphical interface, where all user and admin operations — borrowing, returning, searching, fine viewing, and resource management — are performed through polished GUI windows, eliminating the console dependency and making the system accessible to non-technical users.

7.2 Database Integration

The current flat-file persistence, while functional, is limited in scalability and query capability. A future version will integrate SQLite or MySQL as the storage backend, enabling complex queries,

indexed searches, transaction safety, and support for concurrent multi-user access without file-lock conflicts.

7.3 Expanded User Features and Automation

Future versions will introduce a resource reservation system, membership tier upgrades based on borrowing history or book donations, damage fee deductions on return, an account balance top-up mechanism, and a fully automated notification platform that sends scheduled email reminders for approaching due dates and overdue fine alerts.

7.4 Web Interface

A web-based front-end using a framework such as Flask (Python) or a REST API exposed from the C++ core would allow library members to access the system from any browser, enabling remote borrowing requests, online catalogue browsing, and digital resource downloads.

8. Test Case Runs

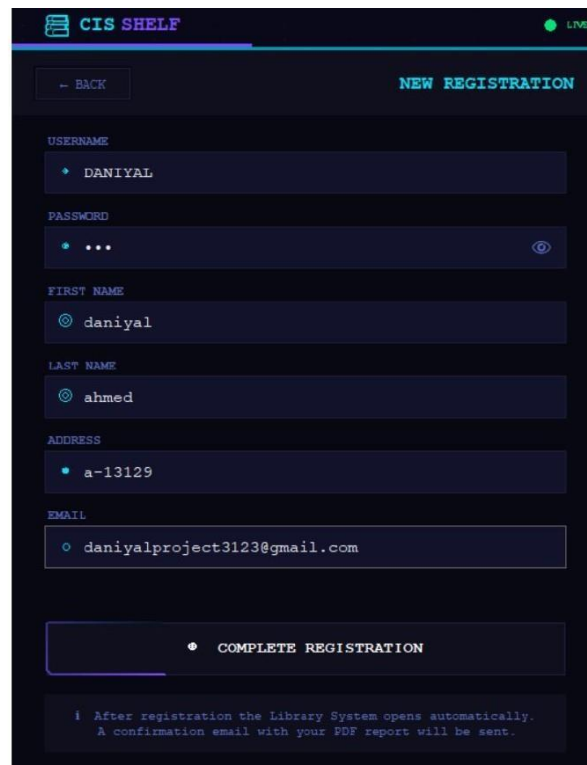
The following screenshots document the system's behaviour across key test scenarios, demonstrating both the graphical and console interfaces as well as the email notification system.

Test Case 1: Graphical Main Menu



Screenshot 1: CIS Shelf Graphical Main Menu — system title, live status indicator, key statistics, and navigation options.

Test Case 2: New User Registration Form

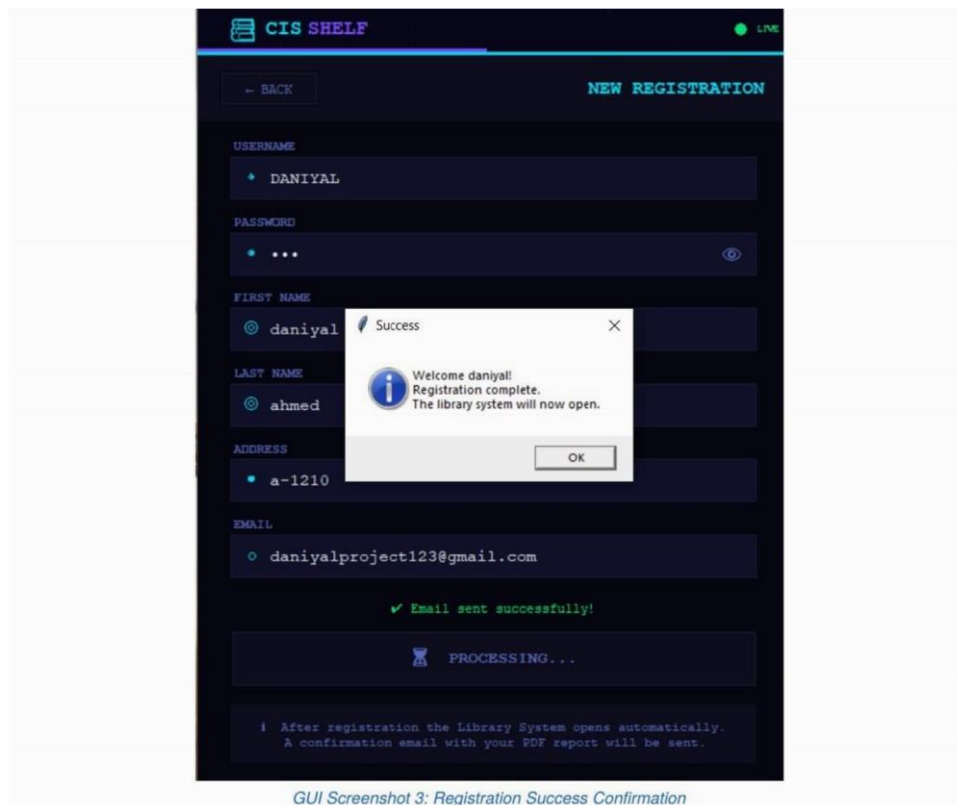


The screenshot shows the 'NEW REGISTRATION' form in the CIS SHELF application. The form includes fields for USERNAME (DANIYAL), PASSWORD (masked with dots and a visibility toggle), FIRST NAME (daniyal), LAST NAME (ahmed), ADDRESS (a-13129), and EMAIL (daniyalproject3123@gmail.com). A 'COMPLETE REGISTRATION' button is at the bottom. A note at the bottom states: 'After registration the Library System opens automatically. A confirmation email with your PDF report will be sent.'

GUI Screenshot 2: New User Registration Form

Screenshot 2: Registration form collecting username, password (with visibility toggle), first name, last name, address, and email.

Test Case 3: Registration Success Confirmation

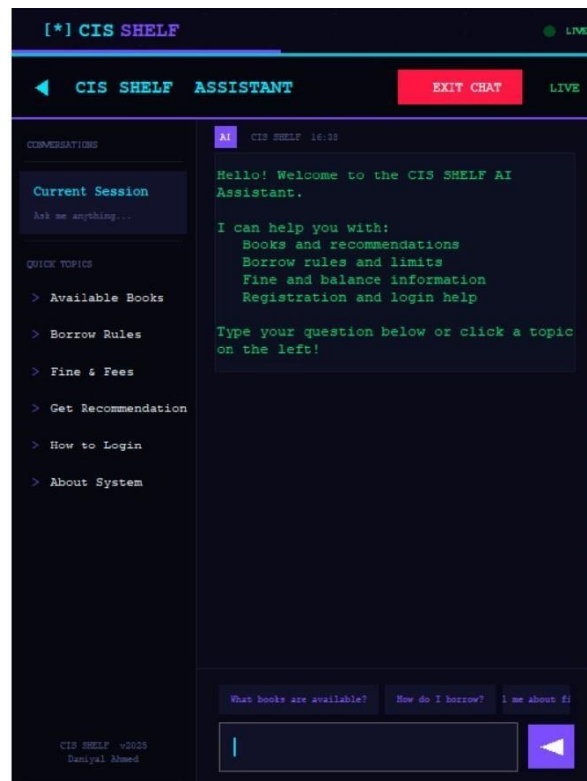


The screenshot shows the 'NEW REGISTRATION' form with a success dialog box overlaid. The dialog box, titled 'Success', contains the message: 'Welcome daniyal! Registration complete. The library system will now open.' with an 'OK' button. The background form shows the same fields as Screenshot 2, but with the ADDRESS field containing 'a-1210' and the EMAIL field containing 'daniyalproject123@gmail.com'. Below the form, a green message states '✓ Email sent successfully!' and a 'PROCESSING...' button is visible. The same note about automatic system opening and email confirmation is at the bottom.

GUI Screenshot 3: Registration Success Confirmation

Screenshot 3: Success dialog confirming registration completion, with email sent confirmation visible in the background.

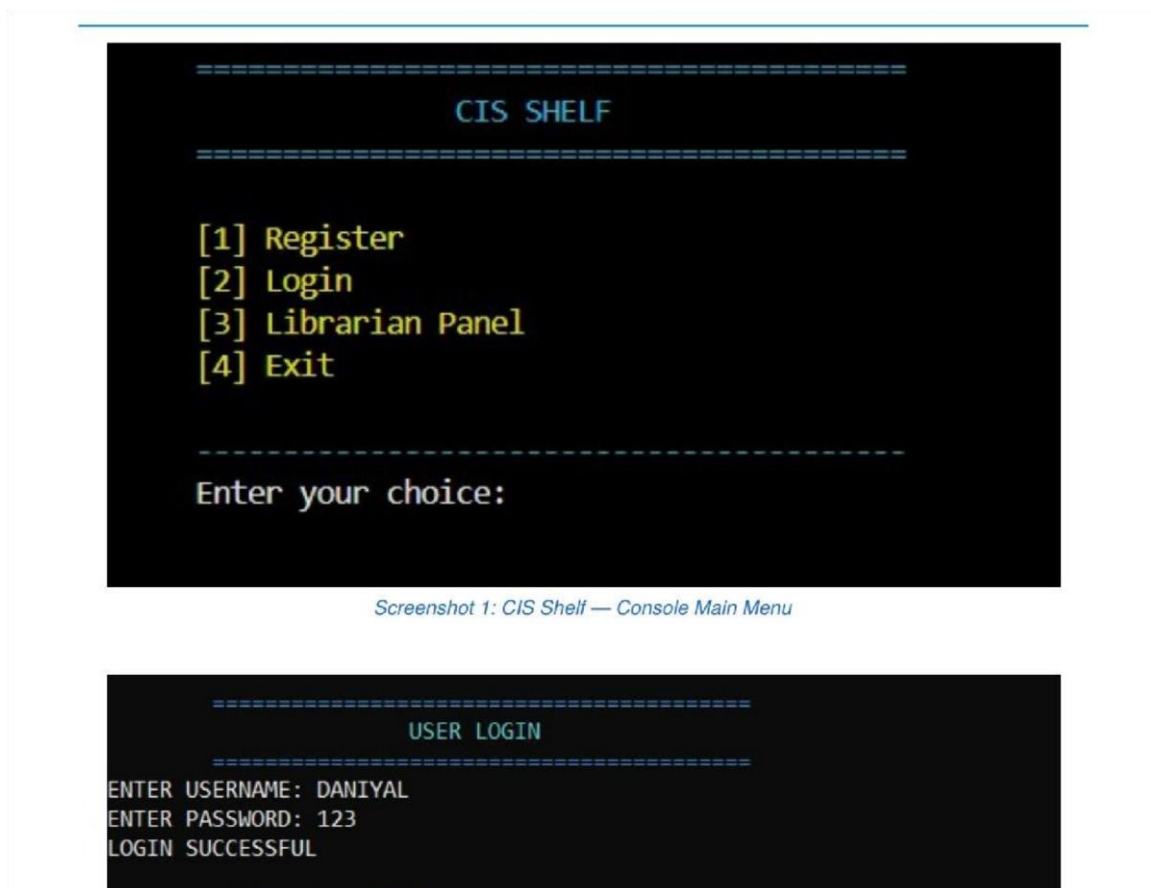
Test Case 4: CIS Shelf AI Assistant



GUI Screenshot 4: CIS Shelf AI Assistant

Screenshot 4: The integrated AI chatbot interface with quick-topic buttons and a free-text input field for realtime library assistance.

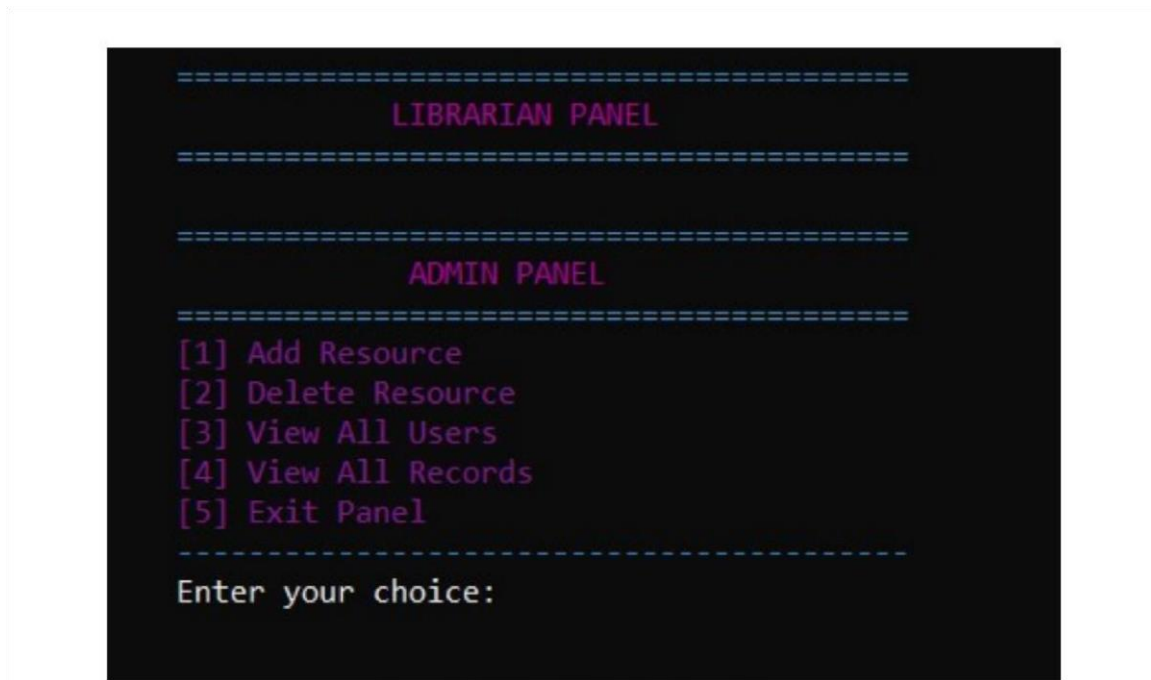
Test Case 5: Console Main Menu and User Login



Screenshot 1: CIS Shelf — Console Main Menu

Screenshot 5: Console main menu (Register / Login / Librarian Panel / Exit) and a successful user login followed by the User Menu.

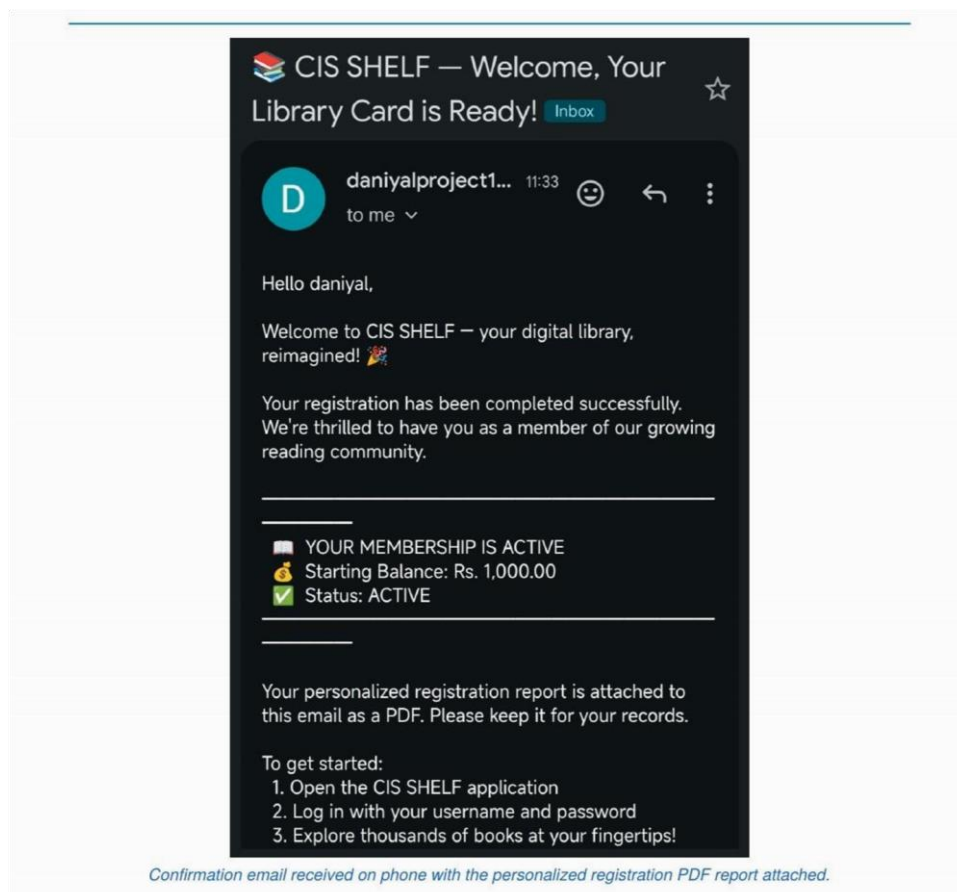
Test Case 6: Librarian Admin Panel



Screenshot 3: Librarian Panel — Admin Menu

Screenshot 6: The Librarian Panel providing administrative operations — Add Resource, Delete Resource, View All Users, View All Records.

Test Case 7: Registration Confirmation Email



Confirmation email received on phone with the personalized registration PDF report attached.

Screenshot 7: Confirmation email received showing membership activation, starting balance (Rs. 1,000), and a personalised PDF report attachment.

9. Languages Used

This project was developed using two programming languages, each serving a distinct and complementary role in the overall system architecture.

9.1 C++ — Core Backend Language

C++ serves as the primary language for all backend logic — class design, file handling, data management, borrowing operations, fine calculation, and menu-driven interactions. Standard libraries used include `<iostream>`, `<fstream>`, `<vector>`, `<ctime>`, `<algorithm>`, `<sstream>`, `<stdexcept>`, and `<limits>`. Windows-specific headers (`windows.h`) are used for console colour output via `SetConsoleTextAttribute`.

9.2 Python — Graphical User Interface and Automation

Python was used to develop the graphical front-end, the email notification system, and the AI assistant chatbot. Python's rich ecosystem of GUI, networking, and PDF libraries made it the ideal complement to the C++ core, bridging back-end logic with a polished user-facing interface.

10. AI Tools Used

The development of CIS Shelf was assisted by the following Artificial Intelligence tools, which were used for guidance, code review, debugging, and documentation.

10.1 Claude by Anthropic

Claude was used as the primary AI assistant for explaining OOP concepts, reviewing class designs, generating structured report content, and producing project documentation. Its contextual understanding of technical problems made it highly effective for both coding support and academic writing tasks.

10.2 ChatGPT

ChatGPT was consulted for brainstorming feature ideas, resolving specific C++ syntax questions, and verifying logic in the borrowing and fine calculation modules.

10.3 Trae

Trae was utilised as an AI-powered coding assistant for real-time code completion, error identification in class implementations, and structural improvement suggestions.

11. Conclusion

The CIS Shelf E-Library Management System successfully demonstrates a comprehensive application of Object-Oriented Programming principles in C++, complemented by a Python-based graphical interface, an AI assistant, and an automated email notification system. The project provided all four team members with invaluable hands-on experience in modular software design, crosslanguage integration, and collaborative engineering — skills that form the foundation of professional software development and that the team will carry forward into future academic and industry projects.